

1 GAP Profiler User Guide

below are instructions on how to use the GAP Profiler parser and GUI. Note that the instructions are meant to be general for all profiles. However, to generate all requisite files, one can navigate to the `examples/` dir and run:

```
source gen_examples.sh
```

making sure that the GAP executable is callable on your machine by just running `gap` (this can be done using the `PATH` env variable). The example used in the GUI is visualized then from `examples/simple_tree/simple_tree.zip`.

1.1 Brief Summary of Views

In the right pane of the interface there are two views of the programs profile, a list view and a tree view. The tree view is fairly intuitive, it just contains the tree of all function calls in the program, and how long they take as a percentage of the total time all of their sibling functions take. For example a function which takes 1 second to run and has 3 identical siblings all taking 1 second would have 25% runtime for that group of calls. The list view is slightly less intuitive. The list view contains 2 levels. The top level of the list view contains pooled information about all calls to a given function over the course of the program. For example, if a function “a” is called 4 times, each taking 1 second, the total time would be 4 seconds (this time does not include time taken by functions called by a). Each of the top level items has one child for each time it is called, which contain information about how long that function took as a percentage of the total time the function took up in all calls across the program. For example, if a function “a” is called 2 times in different parts of a program, each taking 1 second, each would have 50% of the total runtime taken by “a” throughout the course of the program.

An important thing to note here is that the call times in the tree view are the time taken by the function itself, as well as all functions that it calls. The list view excludes time taken by called functions. So if “a” calls “b” the time in the tree view for “a” includes the time taken by “b” and the functions it calls, the list view just includes time taken by lines executed in “a” excluding calls to “b” or any other function.

1.2 Generating a GAP Profile

Before using the parser contained within this project, one must first generate a GAP profile using the built-in profiling package. To do this run these two commands:

```
ProfileLineByLine("output/file.json");
```

before your GAP code is run, and

```
UnProfileLineByLine();
```

after the code has been run. These commands will produce a Json file as output which is then read by the parser module of this project. There are other options for this profiler to generate different profile types. The only kinds which are supported currently by the parser are:

```
ProfileLineByLine("output/file.json", rec(wallTime := true));
ProfileLineByLine("output/file.json", rec(wallTime := false));
ProfileLineByLine("output/file.json", rec(recordMem := true));
```

where each of these options is mutually exclusive. The first option here is the default, recording how much wall clock time (real time) passes while the code is executing. The second option tells the profiler to use the amount of CPU time taken by the program, which should usually be about the same as wall clock time. The third option tells the parser to record the memory allocated per line in place of any data about the time taken by a program. The memory values are treated the same as time values in both this part of the profiler and all parts within this project.

1.3 Parser

The parser is the simpler of the two parts, as it is only ever called from the command line. To run the parser, one must first have generated a Json profile as discussed in the previous section. To convert from a Json profile as generated to a fully contained profile as a Zip archive run the following command:

```
python /path/to/gap_profiler/data_processing/parser.py filename.json
filename.zip path/to/project/root
```

Here the third argument `path/to/project/root` is the path to the root of the GAP project being profiled. This is an optional argument. In addition, the GAP-generated Json file and the profile Zip archive can have different names, but are often given the same name for simplicity. Note also that this command should be run directly after the code so that the source code is in the same state it was in at time of profile generation.

As mentioned the Zip archives are fully self contained, meaning one could generate multiple of these Zip archives and compare them. This is why the Zip archives are separate from the Json files generated by GAP. The Json files contain some data, but become inaccurate as soon as the source code is edited, whereas the Zip archives are valid regardless of circumstance, even if the source code does not exist on the machine.

1.4 Graphical Interface Guide

This is the most complex portion of the project to use. Below is a step by step guide to a basic workflow for the project. All information below is from a profile in the `examples/simple_tree/` directory. The GAP code is shown below:

```
ProfileLineByLine("simple_tree.json");
```

```

d := function()
    Print("Hello World");
end;

b := function()
    d();
end;

c := function()
    d();
end;

# top level function
a := function()
    b();
    c();
end;s

a();

UnprofileLineByLine();
QUIT;

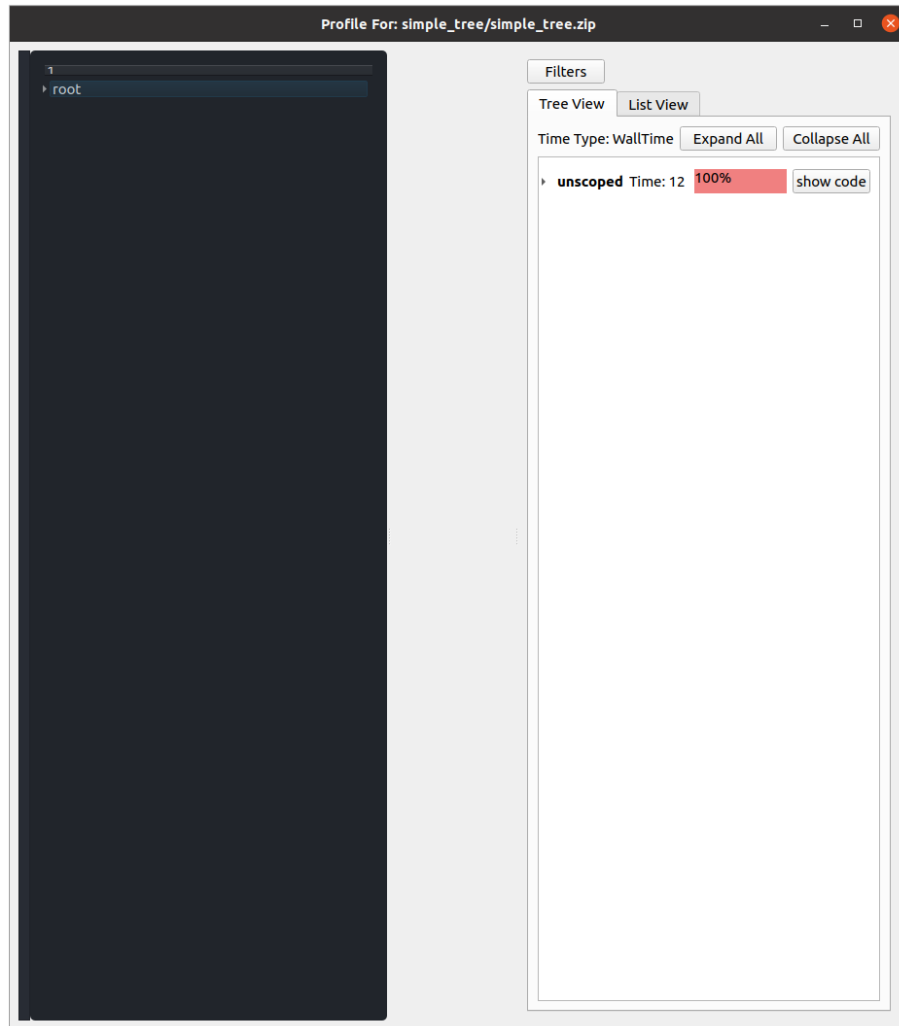
```

1.4.1 Starting the GUI

To start the GUI run:

```
python path/to/gap_profiler/visualization/main.py path/to/profile.zip
```

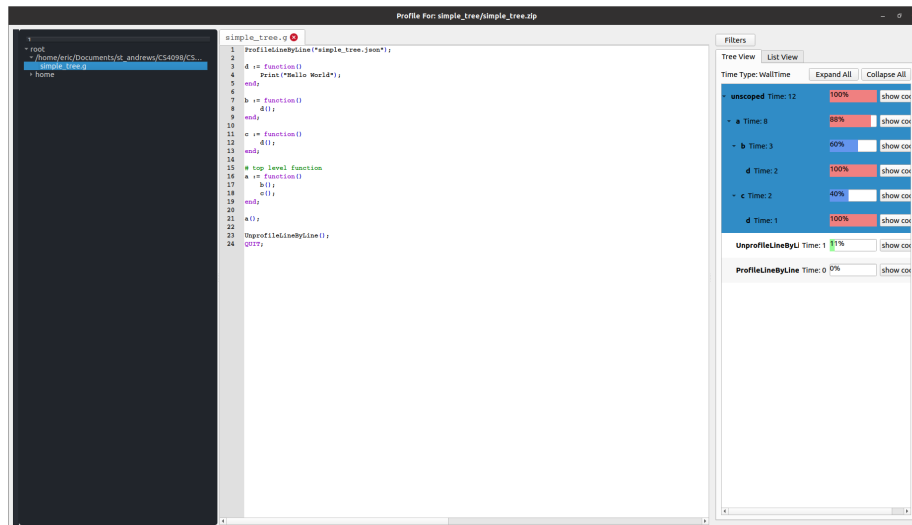
this will bring up the window shown below:



Note that all of the below sections will give instructions starting from this state.

1.4.2 viewing a file

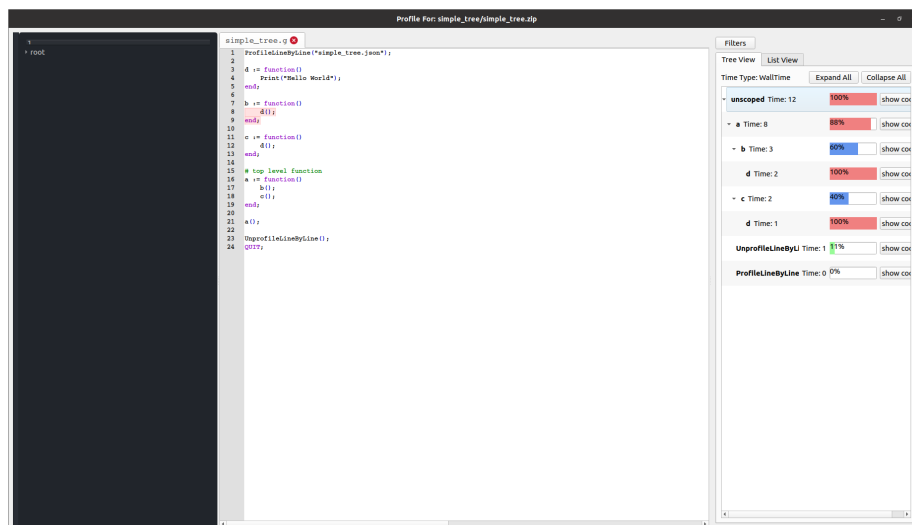
To view a file, click the root item in the left filesystem viewer pane, and continue to expand directories until a source code file is found. Expanding the project root directory (i.e. that passed as the third argument to the parser) should open the file and look like what is shown below:



Notice above that when the file is opened all of the functions in the right pane which are contained within the given file are highlighted.

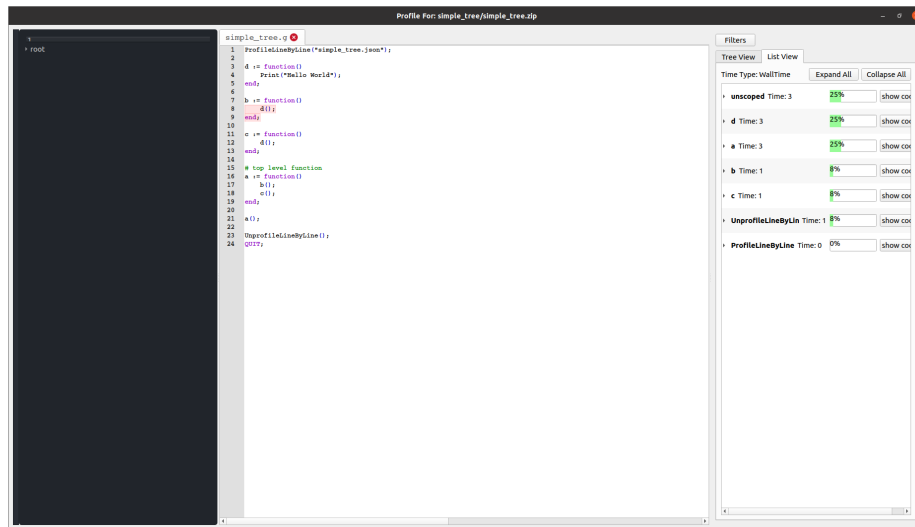
1.4.3 using the show code functionality

Starting again from the profiler state where no files are open, we can click the “Expand All” button in the “Tree View” tab of the right pane. Then, click the “show code” button on the item labelled “b”. You should see the below:



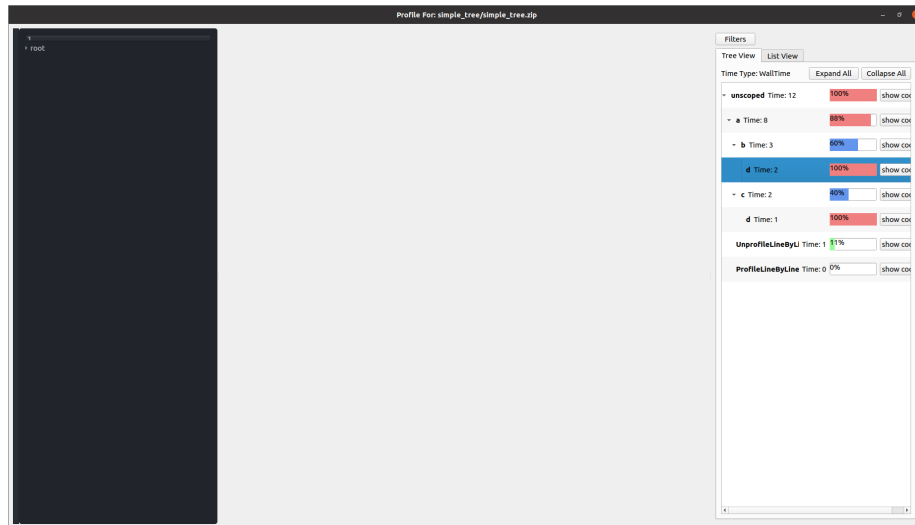
Notice that the file has been opened (even if it was not before) and the code for the function “b” has been highlighted. Note that in GAP the function definition line is not counted as part of the function and is hence not highlighted.

Repeating the same procedure for the “List View” tab gives the below result:

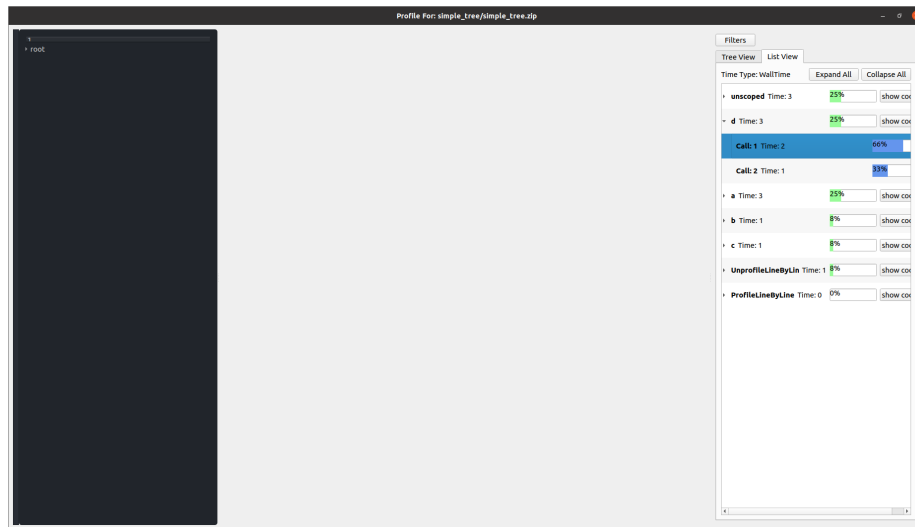


1.4.4 selecting nodes in the view

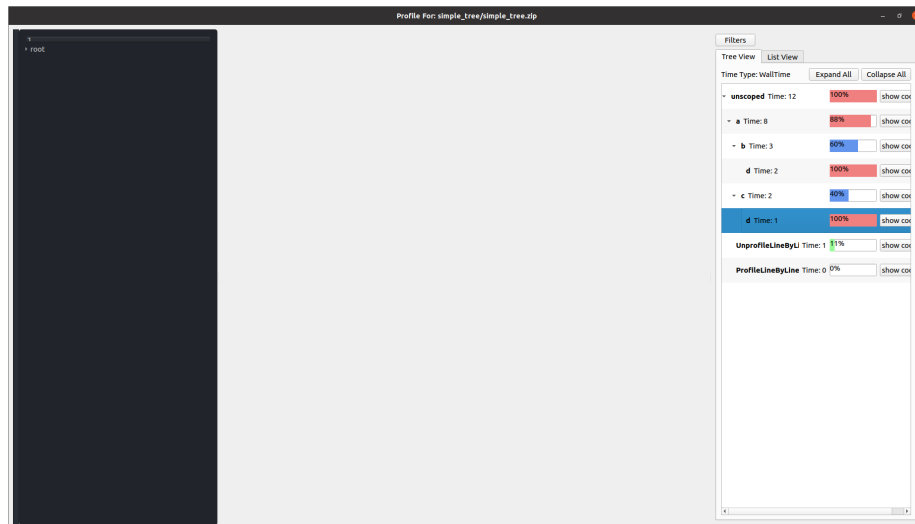
This is a simple bit of functionality which is just to link together the “Tree View” and “List View” tabs. Here, click the “Expand All” button to be able to see all of the items in the “Tree View” tab. Now click the first item in the view labelled “d”. This should yield the below view:

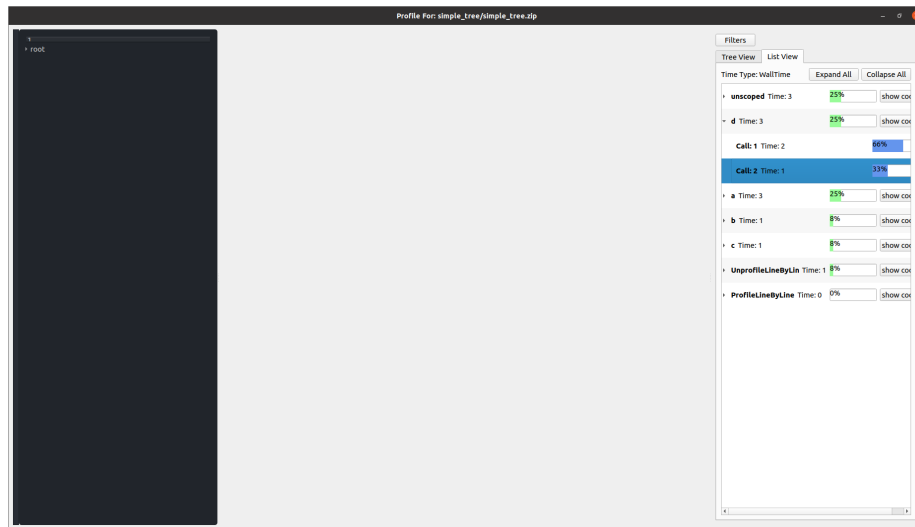


now, if the “List View” tab is clicked to change the view of the profile, the corresponding call in the list view is highlighted as shown below:



Now, returning to the Tree View and clicking the second item labelled “d” and moving back into the list view gives the two states shown below:

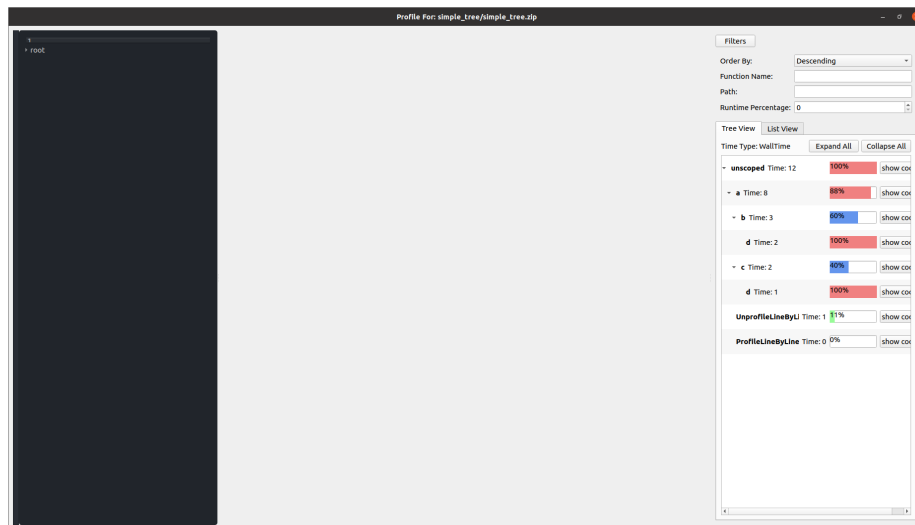


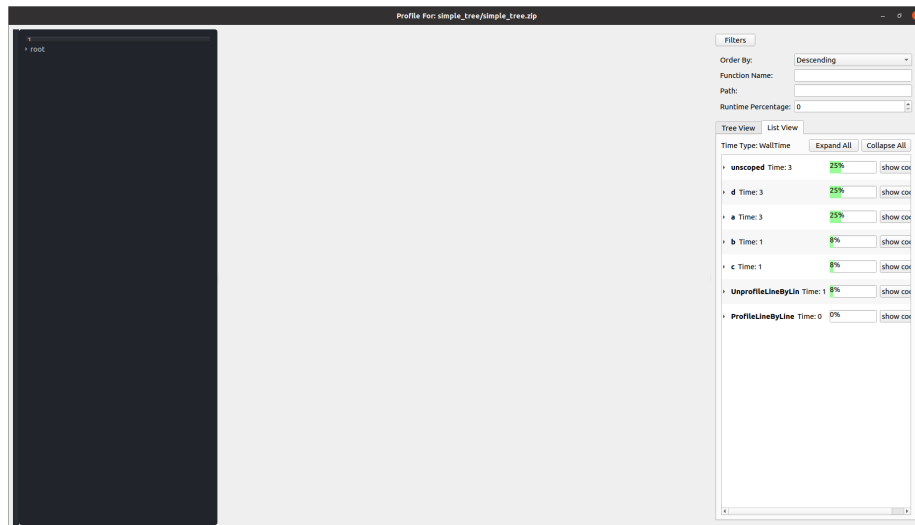


This functionality is simple, and non-invasive, but allows for easy linking between to two profiler viewing tabs. Also note that selecting an item in the list view will highlight its counterpart in the tree view. However this only works for items labelled “Call: x” where x is an integer. This is because the top level items in the list view correspond to functions, not function calls, whereas all items in the “Tree View” tab are individual function calls.

1.4.5 using the filtering interface

To use the filtering interface, first press the “Expand All” button in the “Tree View” tab to make all nodes visible. Next press the “Filters” button to expand the filter view. The result should look like that shown below:

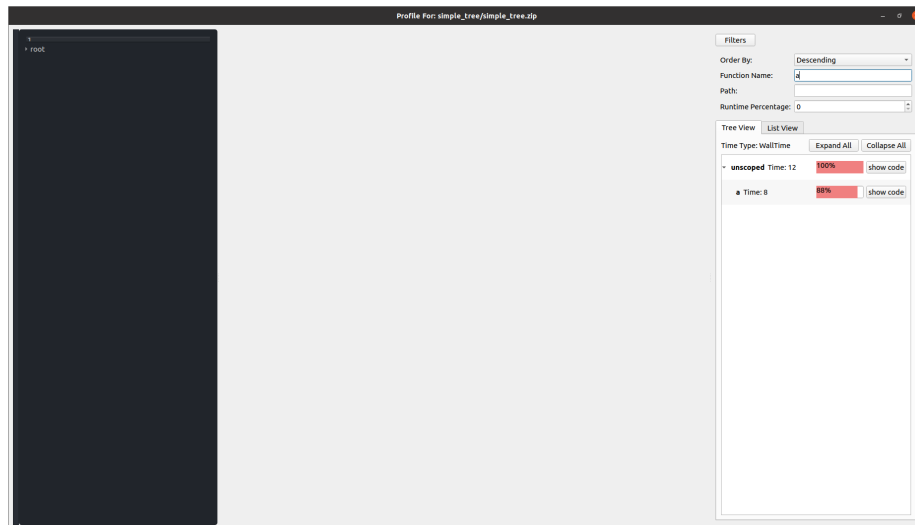


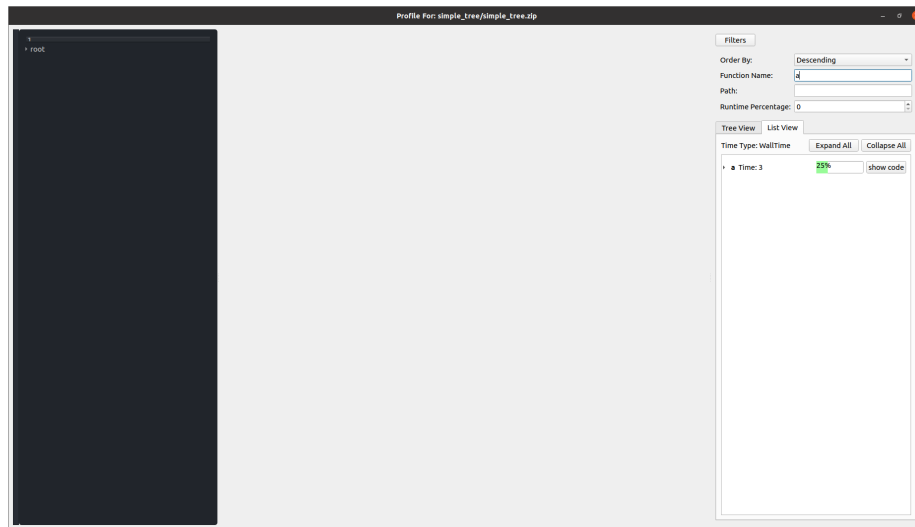


Note that each of these filters can be used in combination with one another, such that an item must pass all filters to be included in the view.

1.4.5.1 function name filtering

Now inputting “a” as our function name filter, we can see that all functions without an “a” in their name are filtered out in the list view and tree view. Note here that parent functions with children (direct or indirect) that have function names containing “a” are not filtered out.

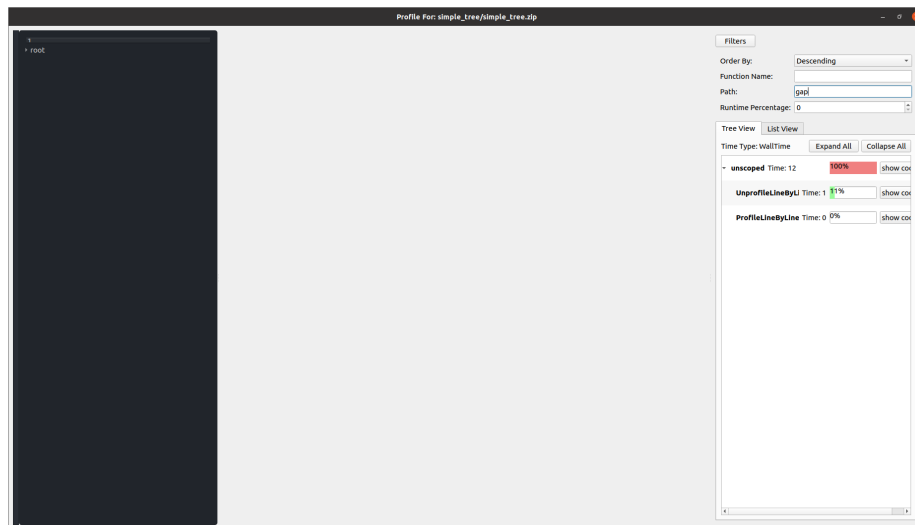


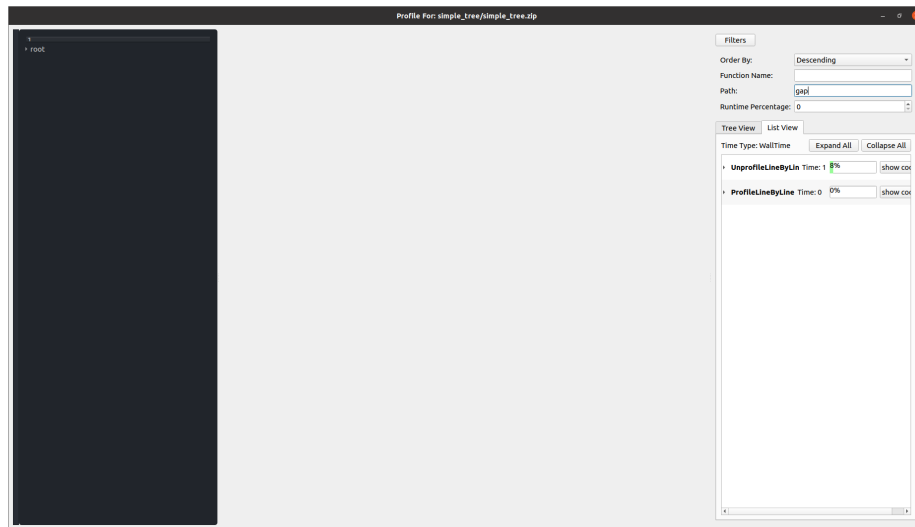


1.4.5.2 path filtering

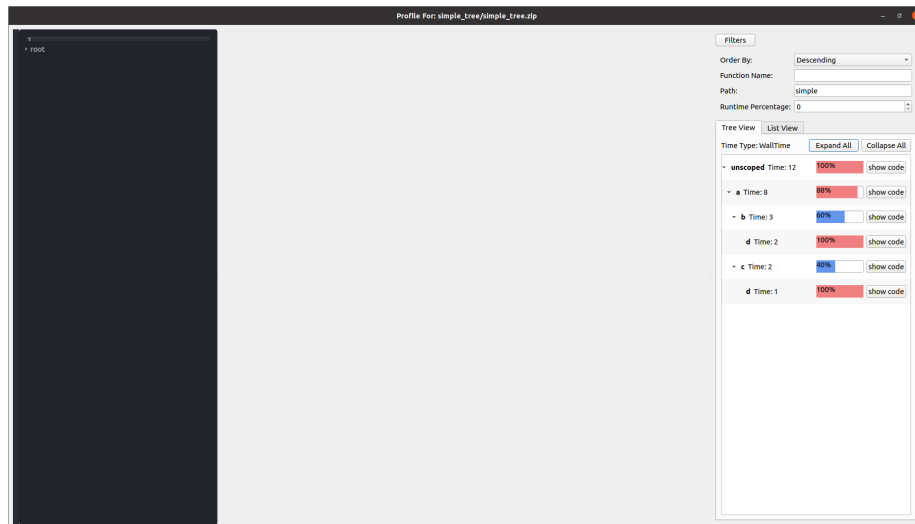
Path filtering is the filtering of functions by their file's paths. This means that if a string is input into the path filtering option which matches part or all of a given function's file path it is included, otherwise it is not. Note that this method of filtering is recursive such that if an item's child is included, it is also included.

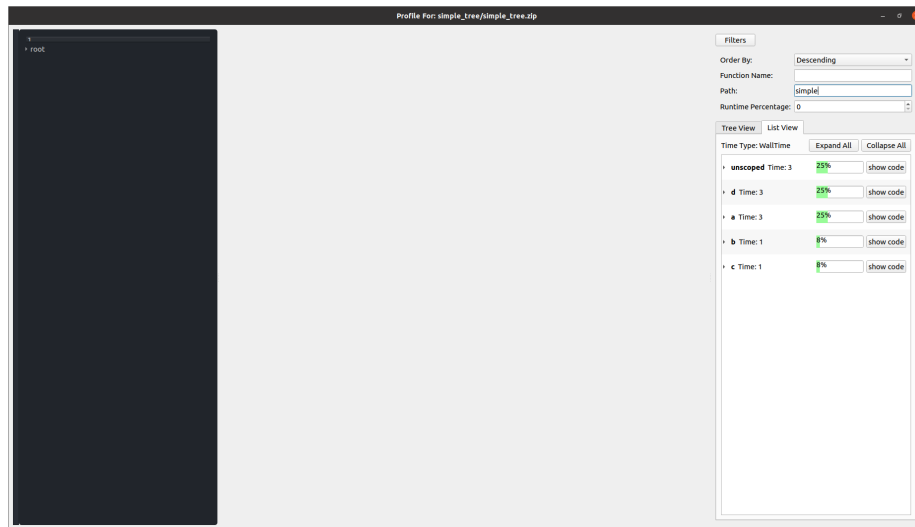
Clearing the filters and now inputting "gap" for the path filter gives us the below result in both views. Notice that this result only shows built-in GAP functionality contained within the GAP package. These functions are just those which generate the profile here.





We can use the path “simple” as well to filter out all of the functions not in the “simple_tree” file or directory. The results of such filtering are shown below:





1.4.5.3 runtime filtering

Here we filter the nodes based on runtime percentage. This method of filtering is non-recursive. This means that if a node has a runtime percentage shown which is less than the input threshold, it and all of its children are filtered out as shown below:

